

COMPREHENSIVE DIGITAL EVIDENCE ACQUISITION AND COMMAND-LINE ANALYSIS USING FTK IMAGER, WINPMEM, AND VOLATILITY

EXPERIMENT NO: 01

DATE:

1. AIM

To perform a complete forensic acquisition and analysis lifecycle by imaging non-volatile storage using AccessData FTK Imager, capturing live volatile memory (RAM) using the WinPmem command-line utility, and analyzing the extracted memory dump using Volatility 3 commands via the PassMark Volatility Workbench.

2. SOFTWARE / TOOLS REQUIRED

- **Operating System:** Windows OS
- **Forensics Tools:**
 - AccessData FTK Imager (GUI for non-volatile physical drive imaging)
 - WinPmem (winpmem_mini_x64_rc2.exe) (CLI tool for volatile memory capture)
 - Volatility 3 Framework (vol.exe / PassMark Volatility Workbench V3.0)
- **Hardware:** Evidence PC/Workstation, Target Storage Drive (USB Flash Disk 31GB USB Device).

3. THEORY

A thorough digital forensic investigation requires collecting both data at rest and data in motion using strict operational procedures.

- **Non-Volatile Memory Acquisition:** AccessData FTK Imager creates mathematically verifiable, bit-by-bit physical images of storage devices (saved as Raw/dd) without altering the original evidence.
- **Volatile Memory (RAM) Acquisition:** Command-line utilities like **WinPmem** load a specialized kernel driver (pme98A2.tmp) to read physical memory directly. Executing this via CMD writes active processes, network connections, and decrypted keys to a raw dump file (.raw).

- **Volatile Memory Analysis (CLI): Volatility 3** is a framework for analyzing RAM dumps. By passing specific parameters and plugins (e.g., -f <image_path> windows.pslist.PsList) into the command line or workbench, investigators can reconstruct the system's live state and identify suspicious processes.

4. PROCEDURE

Part A: Acquiring Non-Volatile Memory (Disk Image) using FTK Imager

Step 1: Select the Source Type

Open FTK Imager as an Administrator. Navigate to **File > Create Disk Image**. In the Select Source dialog, choose **Physical Drive**.

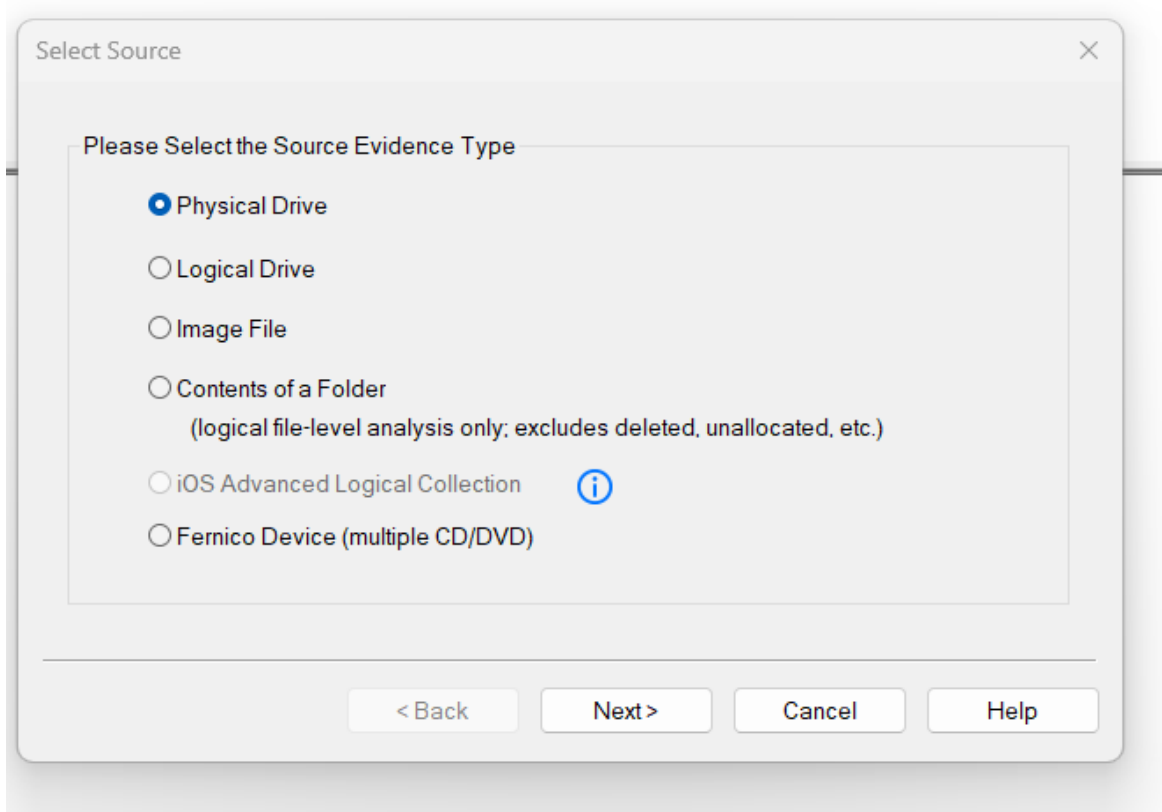


Figure 1: Selecting Physical Drive as the source evidence type

Step 2: Select the Specific Drive

Select the target evidence drive (\\\\.\\PHYSICALDRIVE1 - USB Flash Disk USB Device [31GB USB]) from the dropdown.

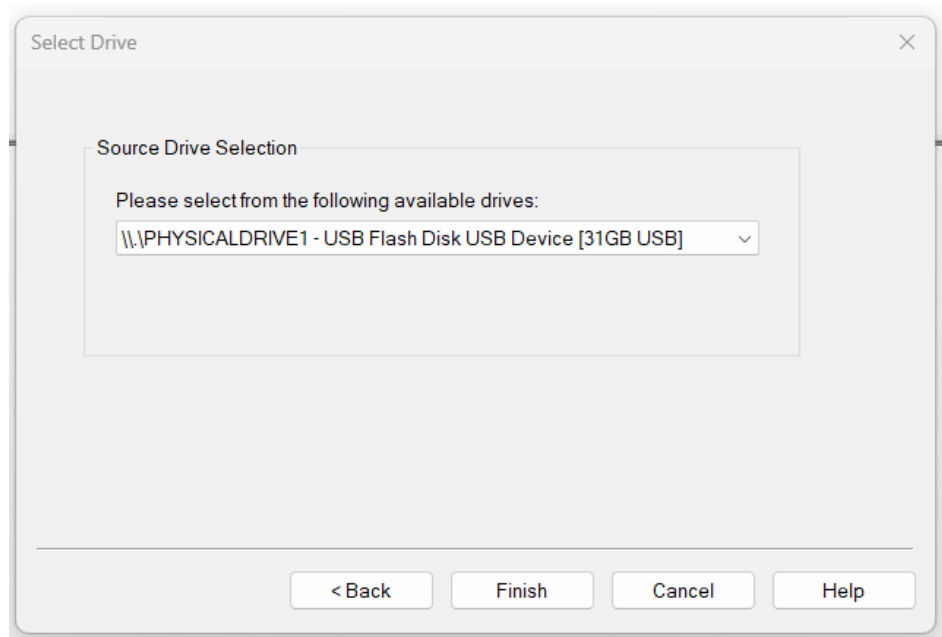


Figure 2: Selecting the target USB device for acquisition

Step 3: Choose the Image Type

Select **Raw (dd)** as the destination image type to create a pure bitstream copy.

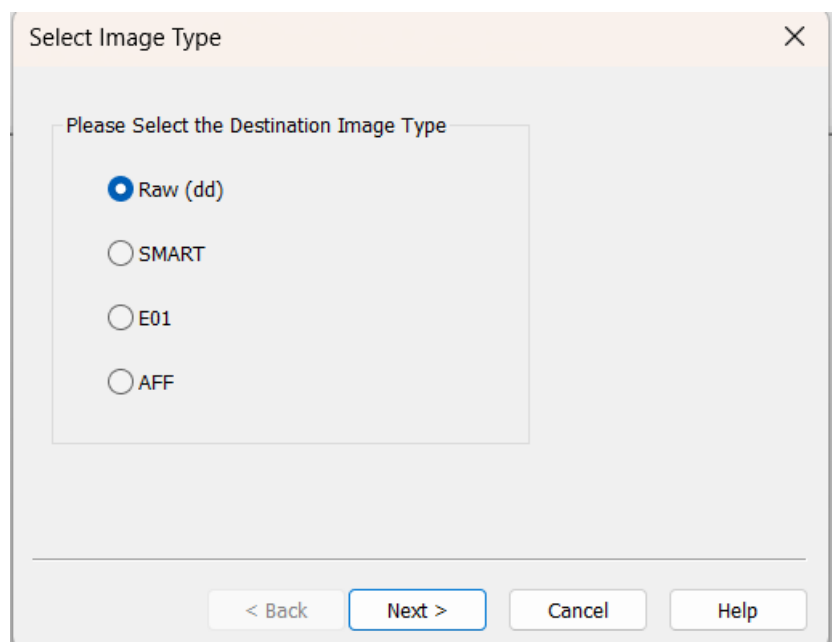


Figure 3: Configuring the destination image format to Raw (dd)

Step 4: Enter Evidence Item Information

Provide the standardized case details to maintain the chain of custody:

- **Case Number:** 99240040007
- **Evidence Number:** 1
- **Unique Description:** Examine
- **Examiner:** Ajvad

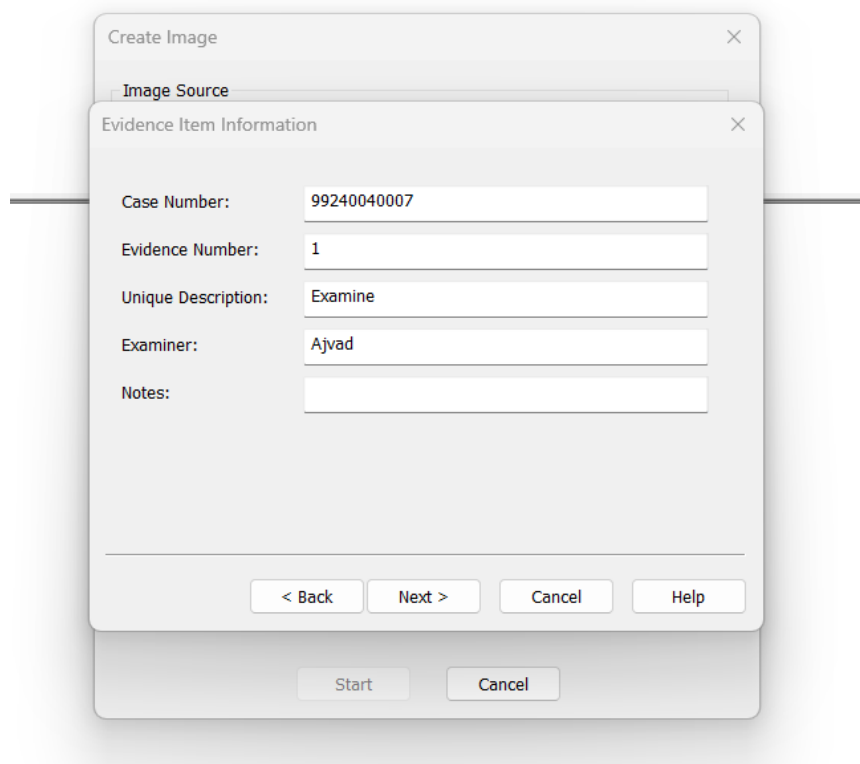


Figure 4: Entering the forensic case and examiner details

Step 5: Configure Image Destination

Set the destination folder to D:\forensics\ftk and the filename to Result. Set the Fragment Size to 20 MB and check **Verify images after they are created**.

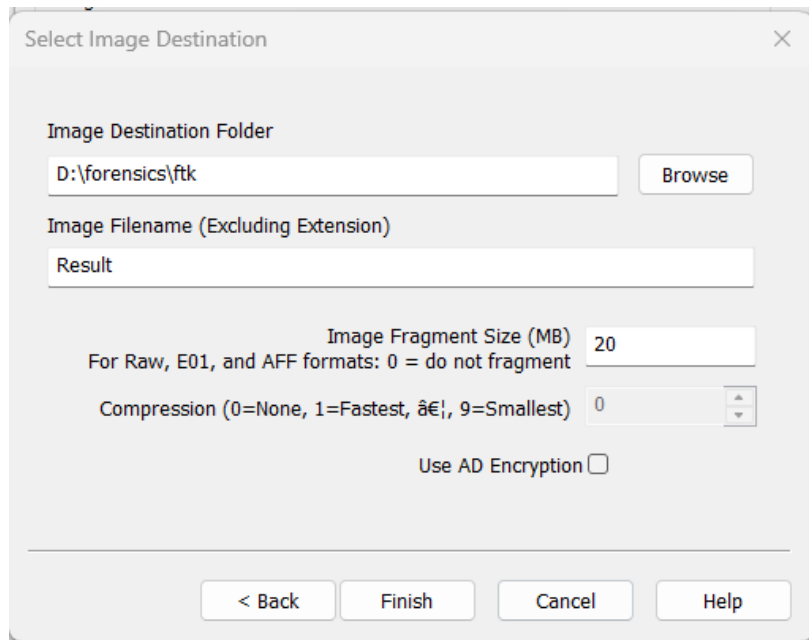


Figure 5: Setting the destination path, filename, and fragmentation size

Step 6: Monitor Acquisition Progress & Hash Verification

Click **Start**. Once complete, FTK Imager hashes the original drive and the newly created image. Review the Drive/Image Verify Results.

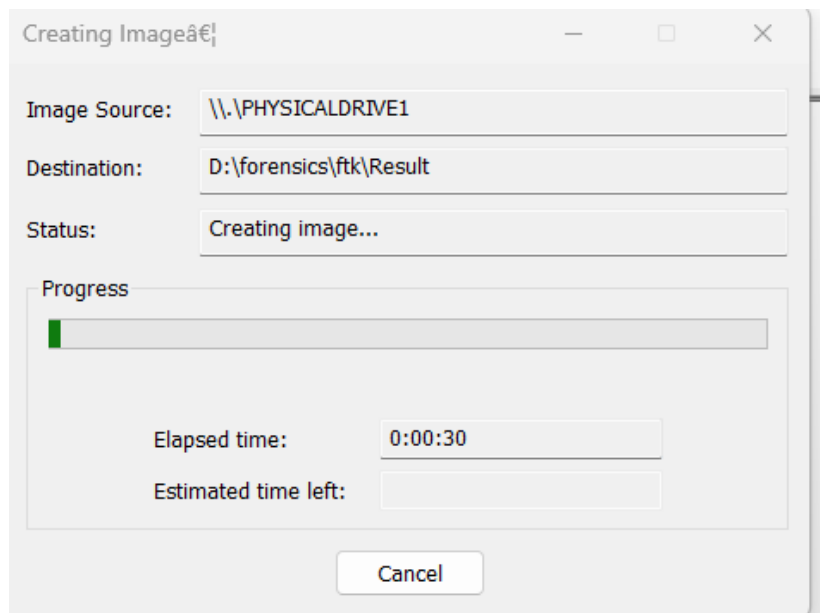


Figure 6: Disk imaging progress tracking

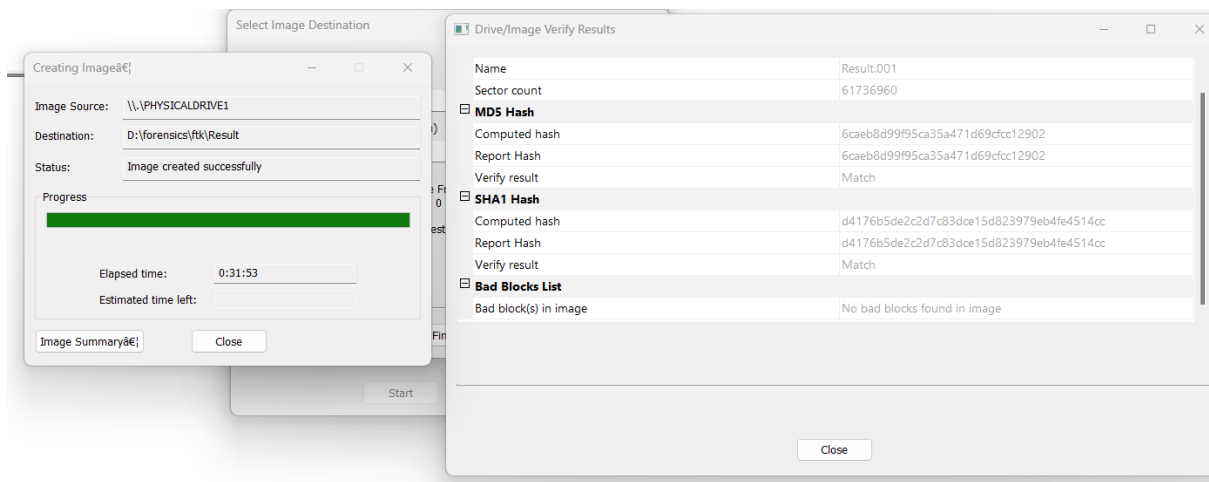


Figure 7: Successful verification of MD5 and SHA1 hashes

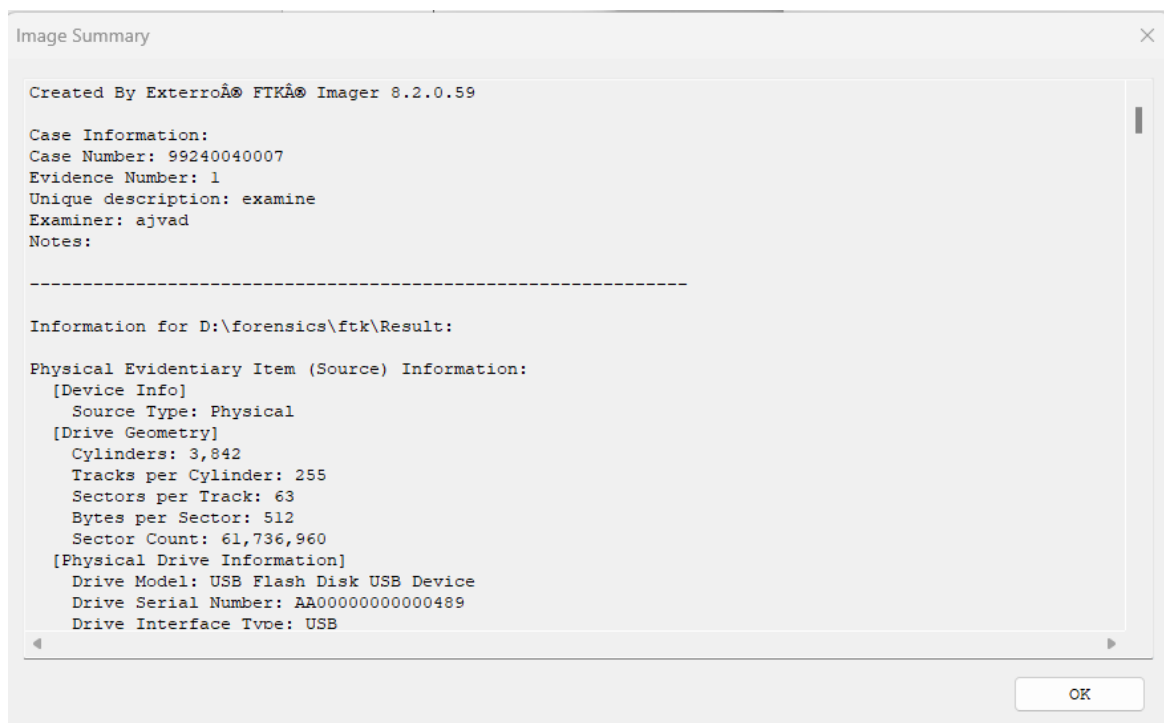


Figure 8: Image Summary detailing the physical device geometry

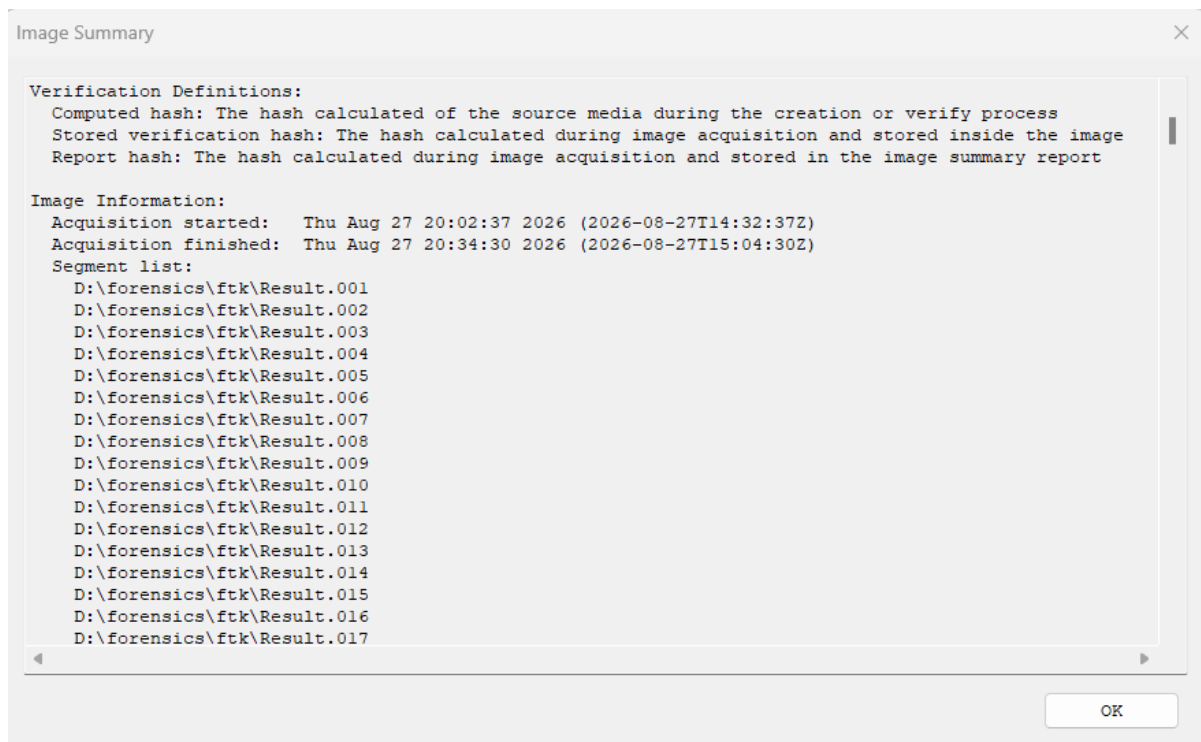


Figure 9: Image Summary detailing the hash verification results and segment list

Part B: Acquiring Volatile Memory (RAM) using WinPmem CMD

Step 1: Launch Command Prompt

Press Win + R, type cmd, and press Ctrl + Shift + Enter to open the Command Prompt as an **Administrator**.

Step 2: Navigate to Directory

Use the cd command to navigate to the folder containing the WinPmem executable:

DOS

A:

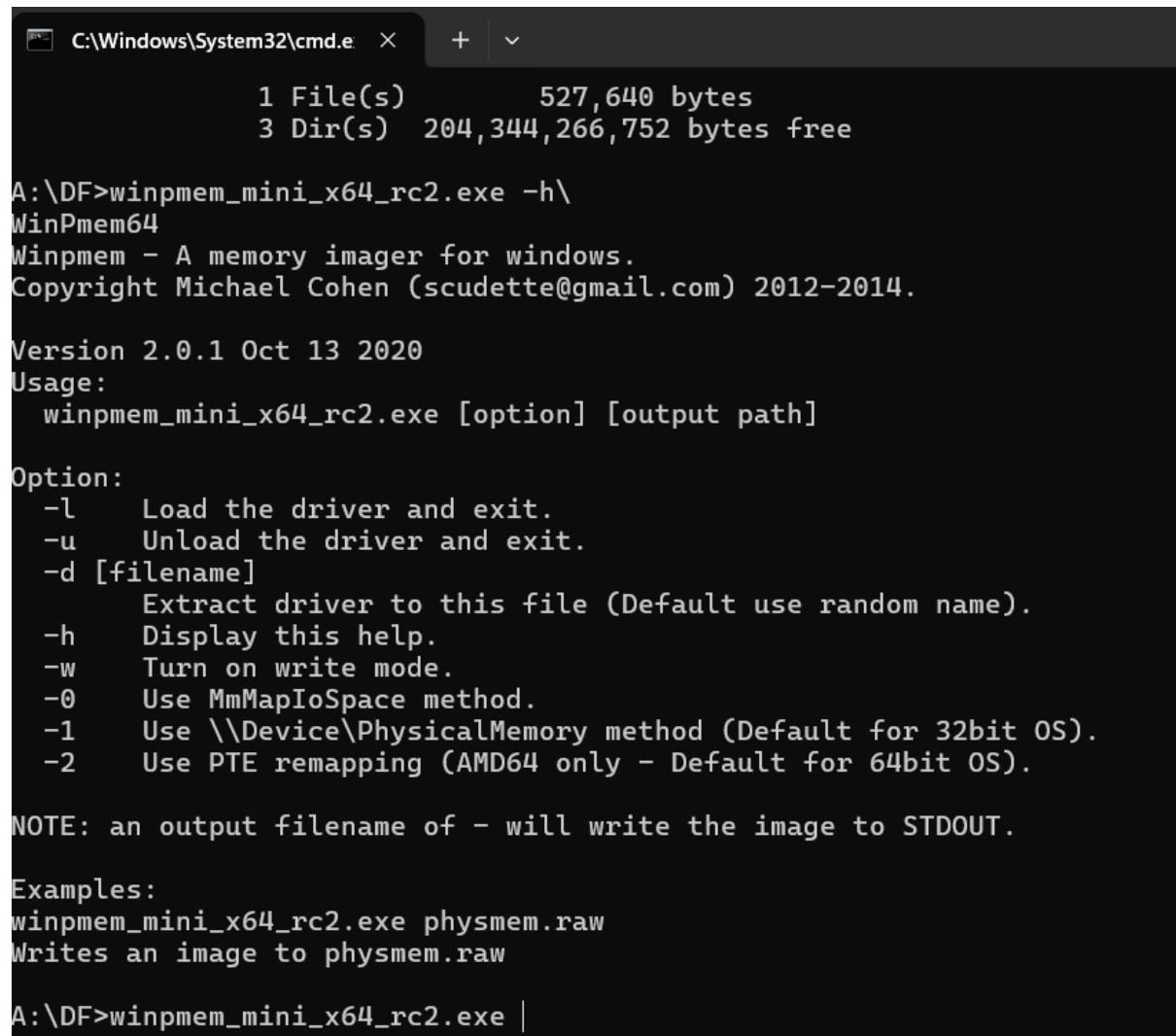
cd \DF

Step 3: Check Available Parameters

Execute the help command to view the WinPmem usage parameters:

DOS

winpmem_mini_x64_rc2.exe -h



```
C:\Windows\System32\cmd.e  X  +  v

1 File(s)          527,640 bytes
3 Dir(s)  204,344,266,752 bytes free

A:\DF>winpmem_mini_x64_rc2.exe -h\
WinPmem64
Winpmem - A memory imager for windows.
Copyright Michael Cohen (scudette@gmail.com) 2012-2014.

Version 2.0.1 Oct 13 2020
Usage:
  winpmem_mini_x64_rc2.exe [option] [output path]

Option:
  -l      Load the driver and exit.
  -u      Unload the driver and exit.
  -d [filename]
          Extract driver to this file (Default use random name).
  -h      Display this help.
  -w      Turn on write mode.
  -0      Use MmMapIoSpace method.
  -1      Use \\Device\PhysicalMemory method (Default for 32bit OS).
  -2      Use PTE remapping (AMD64 only - Default for 64bit OS).

NOTE: an output filename of - will write the image to STDOUT.

Examples:
winpmem_mini_x64_rc2.exe physmem.raw
Writes an image to physmem.raw

A:\DF>winpmem_mini_x64_rc2.exe |
```

Figure 10: Viewing the WinPmem help menu and available parameters

Step 4: Execute Memory Capture Command

Run the memory extraction command, specifying the output filename ajvad.raw. The utility will map the physical memory and write it to the file:

DOS

winpmem_mini_x64_rc2.exe ajvad.raw

```
WinPmem64
Extracting driver to C:\Users\tamil\AppData\Local\Temp\pme98A2.tmp
Driver Unloaded.
Loaded Driver C:\Users\tamil\AppData\Local\Temp\pme98A2.tmp.
Deleting C:\Users\tamil\AppData\Local\Temp\pme98A2.tmp
The system time is: 13:29:47
Will generate a RAW image
- buffer_size_: 0x1000
CR3: 0x00001AE000
4 memory ranges:
Start 0x00001000 - Length 0x0009E000
Start 0x00100000 - Length 0x5CC1E000
Start 0x617FF000 - Length 0x00001000
Start 0x100000000 - Length 0x38F800000
max_physical_memory_ 0x48f800000
Acquisition mode PTE Remapping
Padding from 0x00000000 to 0x00001000
bad
- length: 0x1000

00% 0x00000000 .
copy_memory
- start: 0x1000
- end: 0x9f000

00% 0x00001000 .
Padding from 0x0009F000 to 0x00100000
bad
```

Figure 11: Initializing WinPmem and loading the extraction driver

```

Padding from 0x5CD1E000 to 0x617FF000
pad
- length: 0x4ae1000

07% 0x5CD1E000 .....
copy_memory
- start: 0x617ff000
- end: 0x61800000

08% 0x617FF000 .
Padding from 0x61800000 to 0x100000000
pad
- length: 0x9e800000

08% 0x61800000 .....
08% 0x61800000 .....
08% 0x61800000 .....
08% 0x61800000 .....
copy_memory
- start: 0x100000000
- end: 0x48f800000

21% 0x100000000 .....xxxxx..X.....
26% 0x132000000 .....
30% 0x164000000 .....
34% 0x196000000 .....X.....
39% 0x1C8000000 .....X.....
43% 0x1FA000000 .....
47% 0x22C000000 .....
51% 0x25E000000 .....

```

Figure 12: Memory capture in progress

Step 5: Verification of Dump

Wait for the terminal to display Driver Unloaded. Navigate to the directory to verify the 18.2 GB ajvad.raw file.

```

copy_memory
- start: 0x100000000
- end: 0x48f800000

21% 0x100000000 .....xxxxx..X.....
26% 0x132000000 .....
30% 0x164000000 .....
34% 0x196000000 .....X.....
39% 0x1C8000000 .....X.....
43% 0x1FA000000 .....
47% 0x22C000000 .....
51% 0x25E000000 .....
56% 0x290000000 .....
60% 0x2C2000000 .....
64% 0x2F4000000 .....
69% 0x326000000 .....
73% 0x358000000 .....
77% 0x38A000000 .....
81% 0x3BC000000 .....
86% 0x3EE000000 .....
90% 0x420000000 .....
94% 0x452000000 .....
99% 0x484000000 .....
The system time is: 13:31:38
Driver Unloaded.

A:\DF>

```

Figure 13: Successful completion of the volatile memory capture

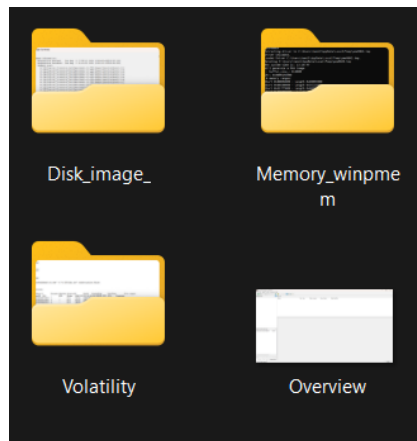


Figure 14: Organized forensic directories for Disk and Memory images

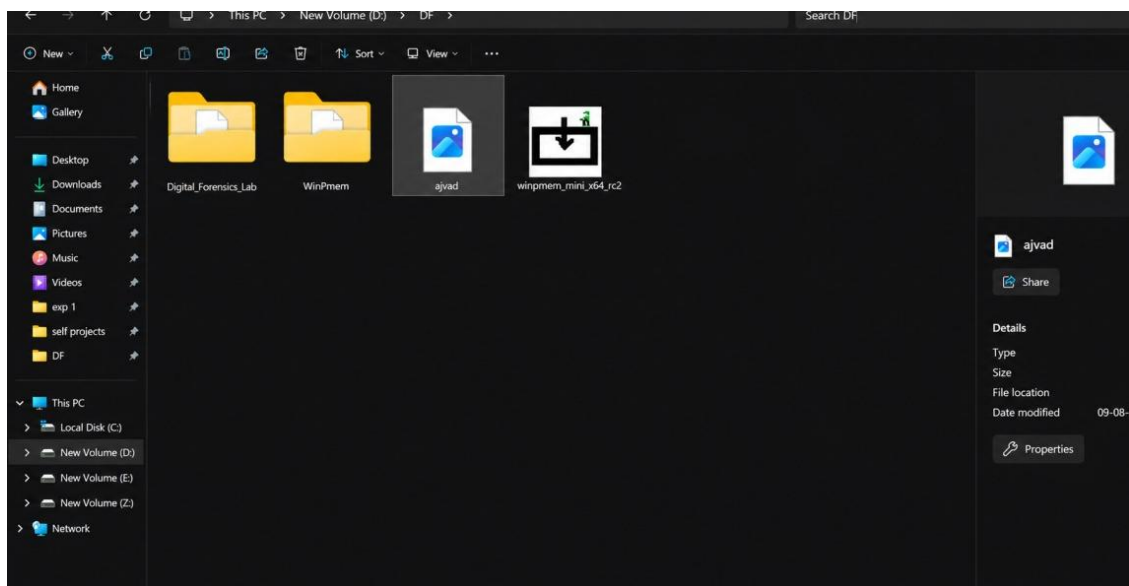


Figure 15: Verification of the generated ajvad.raw memory dump file

Part C: Volatile Memory Analysis using Volatility CLI / Workbench

Step 1: Launch Volatility Environment

Open PassMark Volatility Workbench (which acts as a GUI frontend for vol.exe).

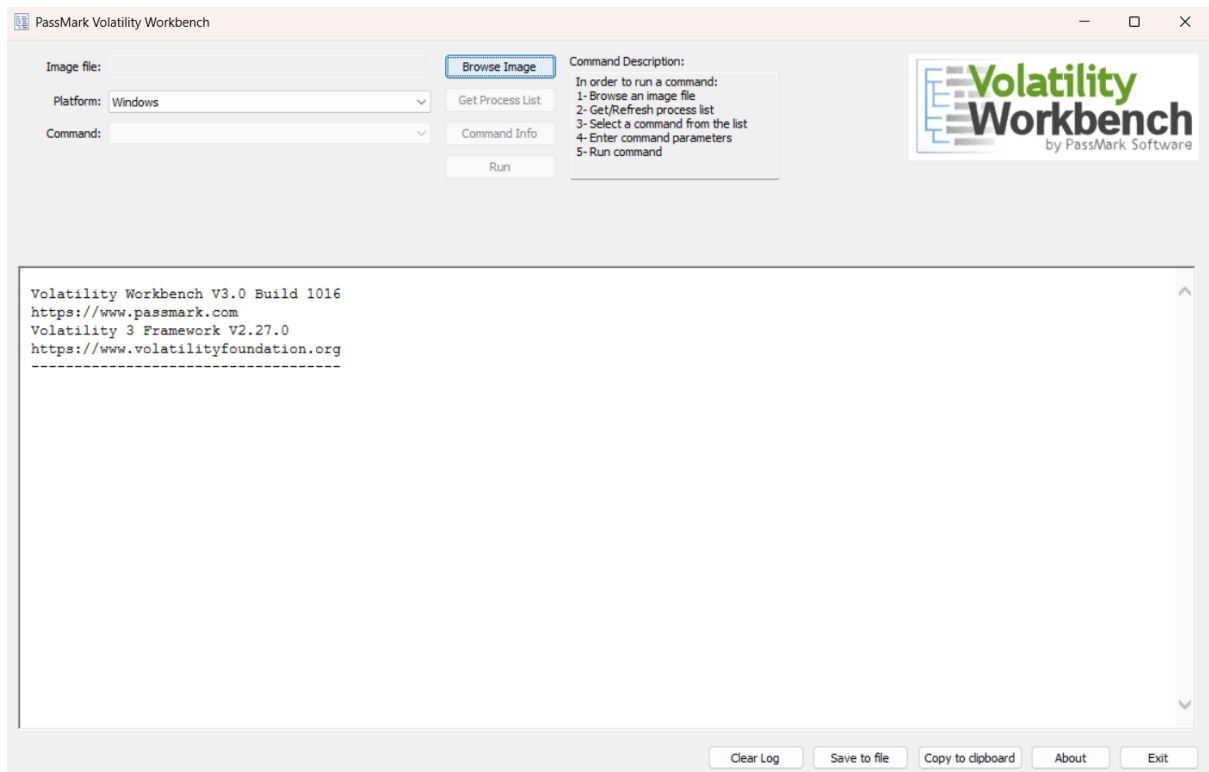


Figure 16: The default interface of PassMark Volatility Workbench

Step 2: Construct and Execute the Volatility Command

To parse the captured memory and list the active processes, load D:\DF\ajvad.raw via **Browse Image**, select the **Windows** platform, choose windows.pslist.PsList from the dropdown, and click **Run**.

The console executes the following command in the background:

DOS

"C:\Users\Downloads\VolatilityWorkbench\vol.exe" -f "D:\DF\ajvad.raw"
windows.pslist.PsList

Or

Use the workbench

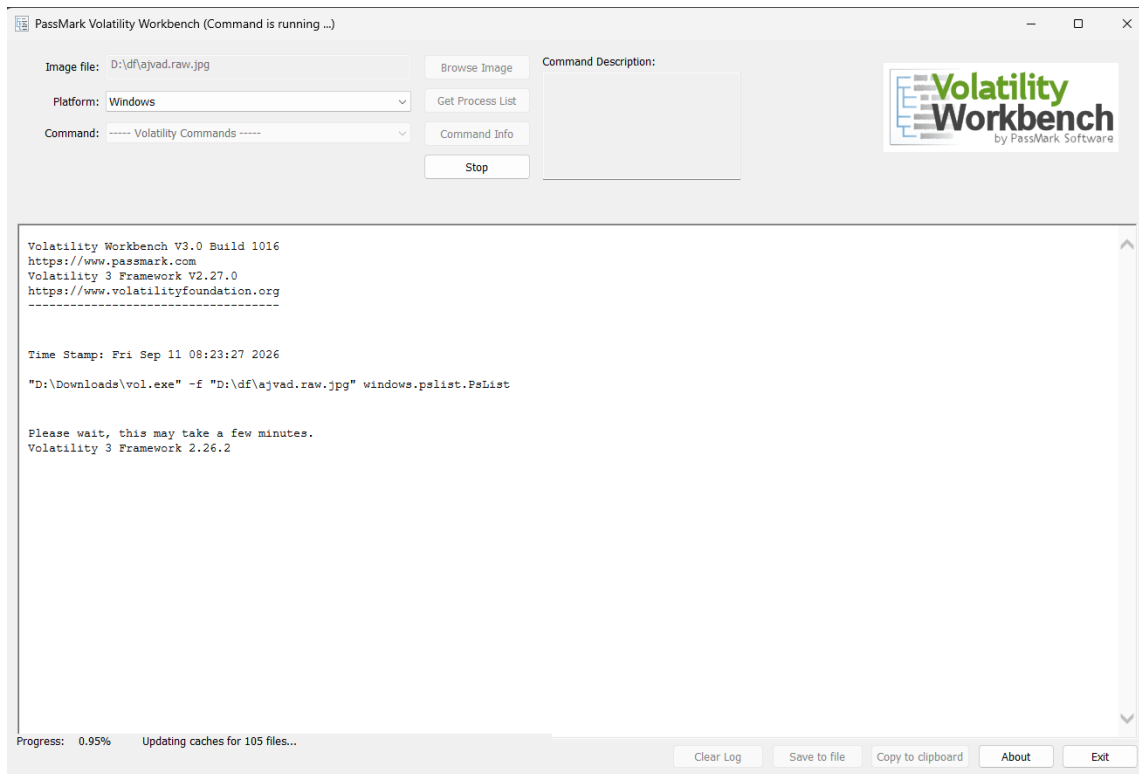


Figure 17: Execution of Volatility for the ajvad.raw image

5. OBSERVATIONS & RESULTS

1. FTK Imager Cryptographic Verification

FTK Imager generated a summary report mathematically confirming the non-volatile image data integrity:

- **MD5 Hash:** 1f8957e555753455d076e144a74afef4 : verified
- **SHA1 Hash:** 613f621a072747853feb89ed81700c749c2cc7b1 : verified

2. Memory Capture & CLI Parsing

The Inba.raw file was successfully written using the winpmem_mini_x64_rc2.exe command. When analyzed, Volatility parsed the _EPROCESS blocks in kernel memory. The resulting output log accurately detailed the Process IDs (PID), Parent Process IDs (PPID), Image File Names, and timestamps of all active programs:

```

Volatility Workbench V3.0 Build 1016
Interface: Win32
Volatility 3 Framework v2.27.0
https://www.volatilityfoundation.org
-----

Time Stamp: Tue Aug 9 19:05:22 2026

"C:\Users\namil\Downloads\Volatility\Workbench\vol.exe" -f "D:\DF\ajvad.raw" windows.pslist.PsList

Please wait, this may take a few minutes...

Volatility 3 Framework 2.26.2
PID PPID ImageFileName Offset(V) Threads Handles SessionId Wow64 CreateTime ExitTime File output
4 0 System 0x8b0de5b9a040 212 0 0 False 2026-09-09 12:12:36.000000 UTC N/A Disabled
180 4 Secure System 0x8b0de599a040 4 0 0 False 2026-09-09 12:23:36.000000 UTC N/A Disabled
228 4 Registry 0x8b0de599a040 4 0 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
240 4 smss.exe 0x8b0de599a040 2 0 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
924 964 csrss.exe 0x8b0de118b040 13 0 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
1068 964 wininit.exe 0x8b0de1e9b040 3 0 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
1068 1048 services.exe 0x8b0de267b010 15 1 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
1120 1048 winlogon.exe 0x8b0de4d1a040 5 1 1 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
1100 1060 userinit.exe 0x8b0de3a28040 7 0 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
1376 1060 lsass.exe 0x8b0de45b0040 17 0 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
1376 1200 vntools.exe 0x8b0de3f58040 21 0 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
1484 1240 WUDFHost.exe 0x8b0de3c5a040 8 0 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
1456 1100 fontdrvhost.exe 0x8b0de43d7040 25 1 0 False 2026-09-09 12:23:46.000000 UTC N/A Disabled
1552 1120 explorer.exe 0x8b0de4a02040 8 0 1 False 2026-09-09 12:23:47.000000 UTC N/A Disabled
1660 1200 svchost.exe 0x8b0de4723040 12 0 0 False 2026-09-09 12:23:47.000000 UTC N/A Disabled
1664 1240 WUDFHost.exe 0x8b0de4888040 25 0 0 False 2026-09-09 12:23:47.000000 UTC N/A Disabled
1804 1182 dm.exe 0x8b0de4c7b040 5 0 0 False 2026-09-09 12:23:47.000000 UTC N/A Disabled
1872 1208 synchost.exe 0x8b0de4f2a040 57 0 0 False 2026-09-09 12:23:47.000000 UTC N/A Disabled

```

Figure 18: Volatility Workbench execution log displaying the extracted Process List

Note: The log accurately shows winpmem_mini_x (PID 19448) actively running, reflecting the acquisition tool executing at the time of the capture.

6. RESULT

The digital evidence acquisition and analysis process was successfully completed. The physical USB storage device was acquired as a Raw (dd) image using FTK Imager, and its integrity was verified using MD5 and SHA1 hash values. Volatile memory was successfully captured using WinPmem through the Command Prompt and saved as a .raw memory dump. The captured memory image was successfully loaded and analyzed using Volatility 3, which extracted the active process information, including PID, PPID, process names, and timestamps. Thus, the required disk imaging, RAM acquisition, hash verification, and memory analysis were successfully performed.